

ASSIGNMENT 6

Instructions

- Create separate .js files for each question.
 - Use descriptive variable names, indentation, and comments to explain logic.
 - Use only core JavaScript (ES6+) — no frameworks.
 - Test using Node.js console or browser console.
 - Use Promises, async/await, callbacks, and error handling wherever required.
-

Q1 – The Startup Morning: Async Coffee Maker

Every morning, the development team starts with a strong cup of coffee. You've been assigned to automate the process virtually! The coffee preparation involves three asynchronous steps — boiling water, brewing coffee, and pouring into a cup.

Task: Implement an Asynchronous Coffee Process

- Each step should be a function returning a Promise that resolves after 1–2 seconds.
 - Use Promise chaining (.then()) to simulate the process.
 - Print logs for each step and finally display:
"Coffee ready for the team!"
 - Use .catch() to handle any failure (simulate random failure using Math.random()).
-

Q2 – Task Scheduler: Micro vs Macro Challenge

You've been debugging a frontend lag issue and need to visualize how the JavaScript Event Loop prioritizes tasks.

Task: Demonstrate Task Order

- Log "Start".
 - Add:
 - One setTimeout() callback (macrotask).
 - One resolved Promise.then() callback (microtask).
 - One synchronous log.
 - Finally, log "End".
 - Run the program and explain (in comments) why microtasks run before macrotasks.
-

Q3 – Bug Tracker: Callback to Promise Migration

Your old bug-tracking code still relies on callbacks. The QA team wants a cleaner, Promise-based solution.

Task: Modernize the Bug Tracker

Original function:

```
function fetchBugs(callback) {  
  setTimeout(() => callback(["UI glitch", "API timeout", "Login failure"]), 1000);  
}
```

- - Convert it into a Promise-based version named getBugs().
 - Log all bugs neatly using console.table().
 - Simulate random API failure using a boolean and handle it using .catch().
-

Q4 – DevOps Delay: Async Timeout Race

Two deployment servers respond at different times. You must track both completion and the first responder.

Task: Manage Concurrent Server Responses

- Server A responds in 2 seconds.
 - Server B responds in 3 seconds.
 - Use `Promise.all()` to show:
"Deployment completed for all servers".
 - Use `Promise.race()` to show:
"Fastest response." for whichever server finishes first.
 - Simulate random failure using `Math.random()` and handle with `.catch()`.
-

Q5 – Frontend Rush: Avoiding Callback Hell

Your UI team has a messy callback chain for its build pipeline. You must rewrite it to improve readability.

Task: Refactor Callback Hell

- The pipeline includes 5 async stages:
design → build → test → deploy → celebrate.
 - Each stage takes 1 second and logs its step.
 - Implement first using nested callbacks (to show callback hell).
 - Then rewrite using `async/await` for cleaner flow.
 - Comment on why `async/await` improves readability.
-

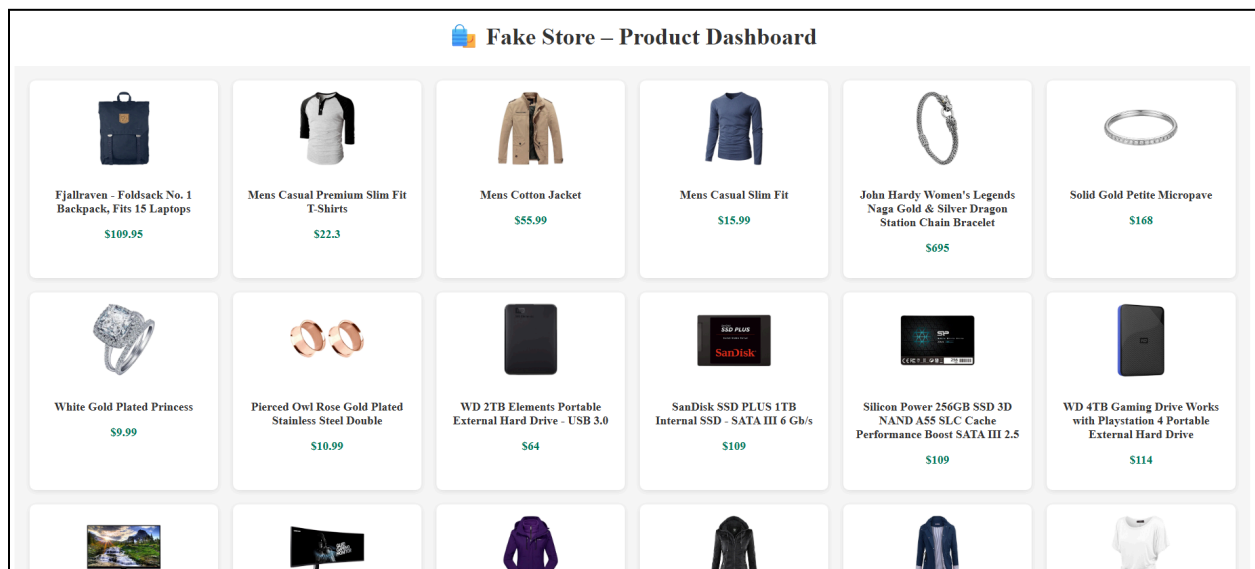
Q6 – E-Commerce Dashboard: Product Card Fetcher

Your company is testing a new mock API for product listings. You need to display product data dynamically using the Fake Store API.

Task: Fetch and Display Product Data

- Use the Fake Store API:
<https://fakestoreapi.com/products>
- Fetch product data using either `fetch().then()` or `async/await`.
- For each product, log:
 - Product Title
 - Price (with \$ prefix)
 - Image URL

Example console output:



Product: Fjallraven Backpack

Price: \$109.95

Image: https://fakestoreapi.com/img/81fPKd-2AYL._AC_SL1500_.jpg

-
- Bonus (Optional): Create HTML product cards dynamically using `document.createElement()` and append to the page.

- Handle errors using try/catch and display "Failed to load products. Please try again." if the request fails.
-

Q7 – The Lazy Loader: Promise Combinator Practice

You're managing a dashboard that loads multiple sections: profile, posts, and messages. Each loads at a different speed.

Task: Use `Promise.allSettled()` for Multiple Async Calls

Use:

```
function loadProfile() { return new Promise(r => setTimeout(() => r("Profile Loaded"), 2000)); }  
function loadPosts() { return new Promise(r => setTimeout(() => r("Posts Loaded"), 1500)); }  
function loadMessages() { return new Promise(r => setTimeout(() => r("Messages Loaded"), 1000)); }
```

- - Use `Promise.allSettled()` to handle them together.
 - Randomly reject one Promise.
 - Print which modules succeeded or failed.
 - Calculate total time taken using `Date.now()` before and after.
-

Q8 – Order Processing Flow: Async Retry Mechanism

Your order submission API often fails randomly. Implement a retry system that attempts multiple times before giving up.

Task: Implement Retry Logic

- Create `submitOrder()` returning a Promise that fails 50% of the time.
- Create an async function `processOrder()` that:

- Tries up to **3 times** using a loop + await.
 - Logs "Attempt X: Success" or "Attempt X: Failed".
 - Throws "Order could not be processed" if all fail.
 - Use try/catch to handle final failure gracefully.
-

Q9 – Debugging the Event Loop

You're analyzing async behavior in JavaScript and want to confirm how microtasks and macrotasks execute.

Task: Predict and Explain Execution Order

Run:

```
console.log("Script start");
setTimeout(() => console.log("Timeout callback"), 0);
Promise.resolve().then(() => console.log("Promise callback"));
console.log("Script end");
```

- - Before running, write your predicted output order in comments.
 - Then execute and compare actual results.
 - Explain why Promises (microtasks) execute before setTimeout (macrotasks).
-

Q10 – The Final Delivery: Async Pipeline Debugger

You are building a food delivery simulation app. Each step of the delivery process happens asynchronously.

Task: Create an Async Delivery Pipeline

- Steps:
takeOrder → prepare → pack → dispatch → deliver
- Each returns a Promise with a random 1–2s delay and success/failure.
- Implement runPipeline() using async/await and try/catch.

Log each step clearly:

Start Pipeline
Step 1: Order taken
Step 2: Food prepared
Step 3: Package ready
Step 4: Out for delivery
Delivery completed!

-
- If any step fails, print "Pipeline failed!".
- Add detailed comments explaining async behavior and event loop control flow.